



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

UNIVERSITI TEKNOLOGI MALAYSIA

Faculty of Electrical Engineering

SEEE3223 — EMBEDDED SYSTEMS

Group Assignment — C Language

Smart Nutrition Scale

Group Members	Student ID
Ali Mohamed Elmagzoub Osman	A24KE3006
MUTAZ MUDATHIR AWAD	A24KE3047
MUSAB JAMALELDIEN MOHAMED YOUSIF	A24KE3015
AWAB AMMAR ISMAIL	A24KE4084

Supervisor: Dr. Fauzan Khairi Che Harun
Session: 2023/2024

Table of Contents

Table of Contents.....	2
1. Introduction	3
1.1 Project Objectives.....	3
2. Program Specification	4
2.1 Library Includes & Global Definitions	4
2.2 Timer 1 Interrupt (Millisecond Time-base).....	4
2.3 GPIO Initialisation	5
2.4 Food Menu Data Structure.....	5
2.5 Setup Function	5
2.6 Main Loop — Display.....	6
2.7 Button Actions	6
3. Program Flowchart.....	7
4. Circuit Diagram	8
4.1 Components List.....	8
4.2 LCD Connection (I2C via PCF8574).....	9
4.3 HX711 Load Cell Amplifier (U2)	9
4.4 Push Buttons	9
4.5 LED Indicator.....	9
4.6 Physical Prototype.....	9
5. Component Configuration	12
5.1 Timer 1 — CTC Mode (Millisecond Interrupt).....	12
5.2 GPIO Pin Configuration	12
5.3 HX711 Scale Calibration.....	13
5.4 LCD Configuration	13
6. Discussion — Subroutine Calls & Interrupt Execution	14
6.1 How the Interrupt Subroutine Works	14
6.2 The delay_ms() Subroutine.....	14
6.3 Subroutine Call Sequence for Each Button.....	14
7. GitHub Repository.....	16
8. Conclusion	17
Appendix A — Full Source Code.....	18

1. Introduction

This report describes the design, implementation, and testing of a Smart Nutrition Scale system developed as part of the SEEE3223 Embedded Systems group assignment. The project utilises a microcontroller platform with multiple input/output peripherals to deliver a practical, real-world embedded application.

The Smart Nutrition Scale allows users to weigh a food item and instantly log its estimated calorie and protein content to a running daily total. The system integrates a load-cell weight sensor, an I2C LCD display, four push-buttons for user interaction, and an LED status indicator — fulfilling the requirement of at least four distinct input/output elements.

The firmware is written in C and compiled using the Arduino framework targeting an AVR microcontroller. Key embedded programming concepts demonstrated include: direct register-level GPIO configuration, hardware timer interrupts for a millisecond time-base, custom delay subroutines, and structured data management using C structs.

1.1 Project Objectives

- Weigh food portions accurately using a HX711-interfaced load cell.
- Display the selected food name and measured weight on a 16×2 LCD.
- Log cumulative daily calories and protein across multiple meals.
- Provide clear button-based navigation (scroll food, confirm, view totals, tare/clear).
- Indicate system activity via an LED and use hardware interrupts for precise timing.

2. Program Specification

This section provides a detailed description of the program functions, broken down by code segment. Each table entry identifies a key instruction or block and explains its role within the embedded system.

2.1 Library Includes & Global Definitions

Code / Instruction	Explanation
<code>#include <Wire.h></code>	Enables I2C communication on the AVR platform, required for the LCD expander (PCF8574).
<code>#include <LiquidCrystal_I2C.h></code>	Provides a high-level API for controlling the 16×2 LCD via the I2C bus.
<code>#include "HX711.h"</code>	Driver library for the HX711 24-bit ADC, used to read the load cell weight sensor.
<code>#include <avr/io.h></code>	Grants direct access to AVR I/O registers (DDRD, PORTD, PIND, etc.) for bare-metal GPIO control.
<code>#include <avr/interrupt.h></code>	Provides <code>ISR()</code> macro and <code>sei()/cli()</code> for enabling hardware interrupts.
<code>LiquidCrystal_I2C lcd(0x27,16,2)</code>	Instantiates the LCD object at I2C address 0x27, configured as 16 columns × 2 rows.
<code>HX711 scale;</code>	Declares the HX711 scale object used throughout the program for weight readings.
<code>#define BTN_CLEAR_BIT (1<<4)</code>	Bitmask for pin D4 — CLEAR button. Uses bit-shift for register-level pin addressing.
<code>#define LED_BIT (1<<0)</code>	Bitmask for pin B0 — Status LED output.
<code>#define LED_ON() / LED_OFF()</code>	Macros that set or clear PB0 using bitwise OR/AND operations directly on PORTB — avoids function call overhead.
<code>#define BTN_x_PRESSED()</code>	Active-low read macros: return true when the corresponding PIND bit is LOW (button pulled to GND).

2.2 Timer 1 Interrupt (Millisecond Time-base)

Code / Instruction	Explanation
<code>volatile uint32_t ms_ticks = 0;</code>	Global counter incremented every 1 ms by the timer ISR. Declared volatile to prevent compiler optimisation across ISR/main boundary.
<code>ISR(TIMER1_COMPA_vect)</code>	Interrupt Service Routine triggered by Timer 1 Compare Match A. Executes every 1 ms and increments <code>ms_ticks</code> .
<code>TCCR1A = 0;</code>	Clears Timer/Counter Control Register A — no PWM modes selected.

<code>TCCR1B = (1<<WGM12) (1<<CS11) (1<<CS10);</code>	Configures Timer 1 in CTC (Clear Timer on Compare) mode with prescaler 64. At 16 MHz: $16,000,000 / 64 = 250,000$ ticks/s.
<code>OCR1A = 249;</code>	Compare value that triggers the ISR. $250,000 / 250 = 1,000$ interrupts per second $\rightarrow$ 1 ms period.
<code>TIMSK1 = (1<<OCIE1A);</code>	Enables the Timer 1 Output Compare A interrupt in the Timer Interrupt Mask Register.
<code>sei();</code>	Global interrupt enable — activates all unmasked interrupts.
<code>void delay_ms(uint32_t ms)</code>	Busy-wait delay subroutine. Reads <code>ms_ticks</code> at the start, then loops until the elapsed difference equals the requested delay. Non-blocking to other ISRs during the wait.

2.3 GPIO Initialisation

Code / Instruction	Explanation
<code>DDRD &= ~(BTN_CLEAR_BIT ...);</code>	Configures D4–D7 as inputs by clearing the corresponding bits in DDRD (Data Direction Register D).
<code>PORTD = (BTN_CLEAR_BIT ...);</code>	Enables internal pull-up resistors on D4–D7 so the lines default HIGH; buttons pull them LOW when pressed.
<code>DDRB = LED_BIT;</code>	Configures PB0 as an output by setting bit 0 in DDRB.
<code>PORTB &= ~LED_BIT;</code>	Initialises the LED output LOW (off) at startup.

2.4 Food Menu Data Structure

Code / Instruction	Explanation
<code>typedef struct { char name[10]; float cal_per_g; float pro_per_g; } FoodItem;</code>	Defines a <code>FoodItem</code> struct storing the display name and per-gram calorie/protein values for a single food entry.
<code>FoodItem menu[MENU_SIZE]</code>	Array of 8 pre-defined food items (Chicken, Rice, Eggs, Beef, Broccoli, Oats, Whey Protein, Salmon) with their respective nutritional constants.
<code>int menu_idx = 0;</code>	Index of the currently selected food item. Incremented by <code>BTN_SCROLL</code> and wraps at <code>MENU_SIZE</code> .
<code>float daily_cal, daily_pro;</code>	Accumulator variables for the user's total calories and protein logged during the session.

2.5 Setup Function

Code / Instruction	Explanation
--------------------	-------------

<code>gpio_init(); timer1_init();</code>	Calls both initialisation subroutines to configure hardware before entering the main loop.
<code>lcd.init(); lcd.backlight();</code>	Initialises the LCD over I2C and activates the backlight.
<code>scale.begin(DT_PIN, SCK_PIN);</code>	Initialises the HX711 driver with the assigned data (D2) and clock (D3) pins.
<code>scale.set_scale(4.2);</code>	Sets the calibration factor (4.2) to convert HX711 raw ADC counts to grams. Value obtained during physical calibration.
<code>scale.tare();</code>	Zeros the scale reading to account for any container or offset weight.

2.6 Main Loop — Display

Code / Instruction	Explanation
<code>current_weight = scale.get_units(3);</code>	Reads the average of 3 HX711 samples and converts to grams using the calibration factor. Averaging reduces noise.
<code>if (current_weight < 0) current_weight = 0;</code>	Clamps negative readings (drift near zero) to zero for a cleaner display.
<code>lcd.setCursor(0,0); lcd.print(...);</code>	Updates the top row with the current food name and the bottom row with the measured weight — refreshed every loop iteration.

2.7 Button Actions

Code / Instruction	Explanation
<code>BTN_CLEAR_PRESSED()</code>	Tares (re-zeros) the scale, displays confirmation message for 1 s, turns LED on during the operation.
<code>BTN_VIEW_PRESSED()</code>	Shows running totals (daily calories and protein) on the LCD. If held for ≥ 2 s, resets both accumulators to zero and confirms with a message.
<code>BTN_SCROLL_PRESSED()</code>	Advances <code>menu_idx</code> by 1, wrapping from the last item back to index 0. 300 ms debounce delay prevents multiple triggers per press.
<code>BTN_CONFIRM_PRESSED()</code>	Computes added calories and protein ($\text{current_weight} \times \text{per-gram constants}$), adds to running totals, and displays a confirmation with updated totals for 4 s.

3. Program Flowchart

The flowchart below illustrates the overall program flow, from system initialisation through the continuous main loop and each button-triggered action.

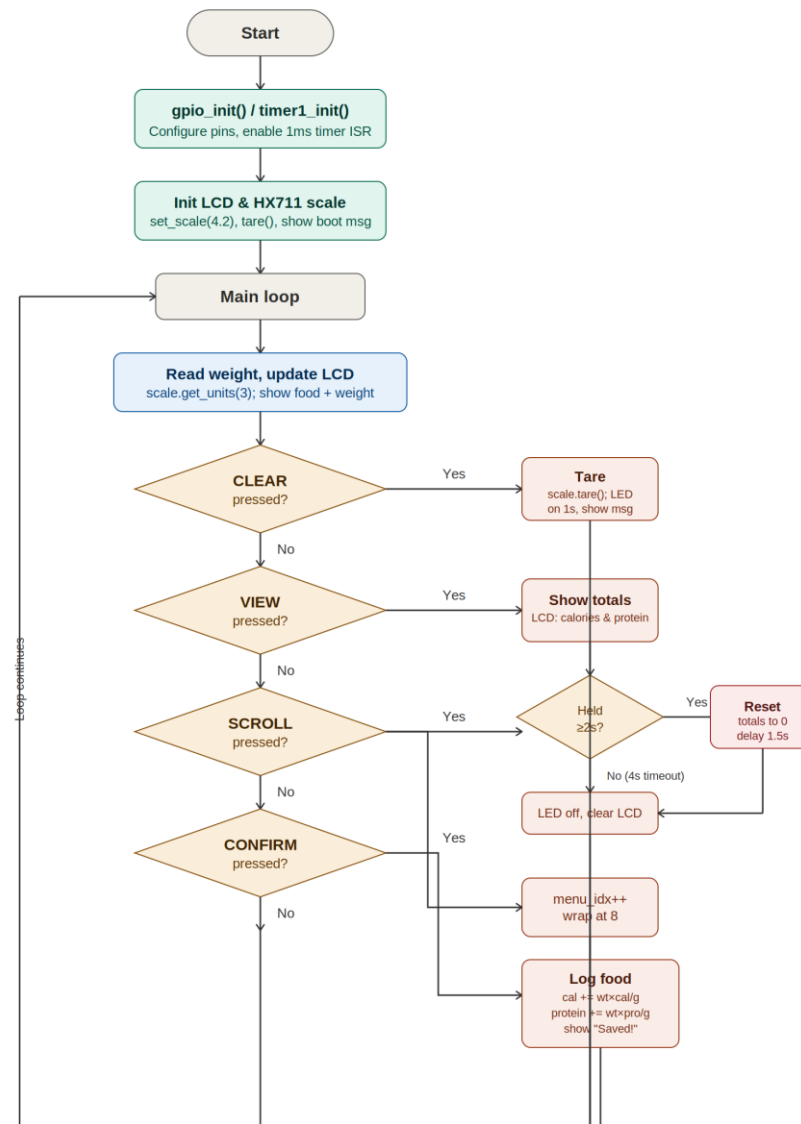


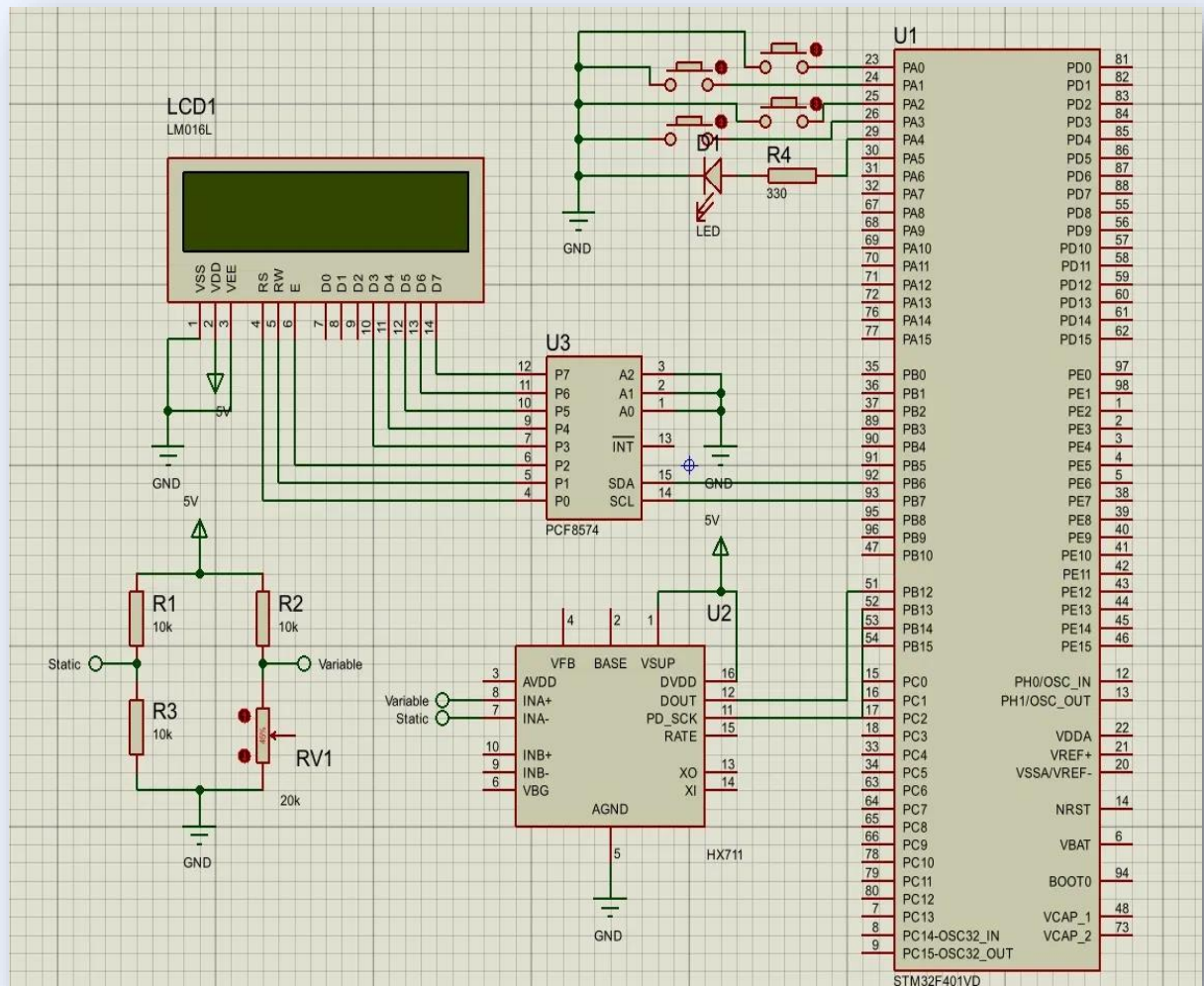
Figure 3.1 — Complete System Flowchart

The program begins by calling `gpio_init()` and `timer1_init()` to configure hardware peripherals, followed by LCD and HX711 setup. The main loop runs continuously, reading weight and refreshing the display. Each iteration polls the four buttons; the corresponding action is executed immediately upon detection of an active-low input.

The `BTN_VIEW` button implements a dual-function press: a short press displays totals for 4 s, while a 2-second hold triggers a data reset. This is implemented via an inner polling loop that tracks elapsed hold time using the `ms_ticks` time-base.

4. Circuit Diagram

The circuit was designed to interface all required peripherals with the microcontroller. The schematic was produced using Proteus Design Suite and is shown below.



Key circuit elements are described in the sections that follow.

4.1 Components List

Component	Type	Interface / Connection
LCD (I2C LM016L)	Display	I2C (SDA/SCL) via PCF8574
HX711 Load Cell Amplifier	Sensor Interface	Digital (DOUT → D2, SCK → D3)

Push Button × 4	Input	Digital GPIO (D4, D5, D6, D7) — active-low w/ pull-up
LED × 1	Output	Digital GPIO (B0) with 330 Ω series resistor
Load Cell / Weight Sensor	Sensor	Analog differential → HX711 ADC → MCU

4.2 LCD Connection (I2C via PCF8574)

The LM016L 16×2 character LCD is connected through a PCF8574 I2C I/O expander (U3). The expander reduces the required MCU pins from 6 (parallel) to 2 (SDA, SCL). The I2C bus operates at 100 kHz with the address 0x27. A 5 V supply powers both the LCD and the expander. The contrast potentiometer is connected between VDD and VSS with the wiper feeding the VEE pin.

4.3 HX711 Load Cell Amplifier (U2)

The HX711 (U2) receives differential signals from a full-bridge strain-gauge load cell on its INA+/INA- inputs. It amplifies and digitises the signal with 24-bit resolution and transmits data serially to the MCU. DOUT is connected to D2 and PD_SCK to D3. The chip is powered by the 5 V rail; DVDD drives the internal reference. The RATE pin is tied LOW for 10 samples per second.

4.4 Push Buttons

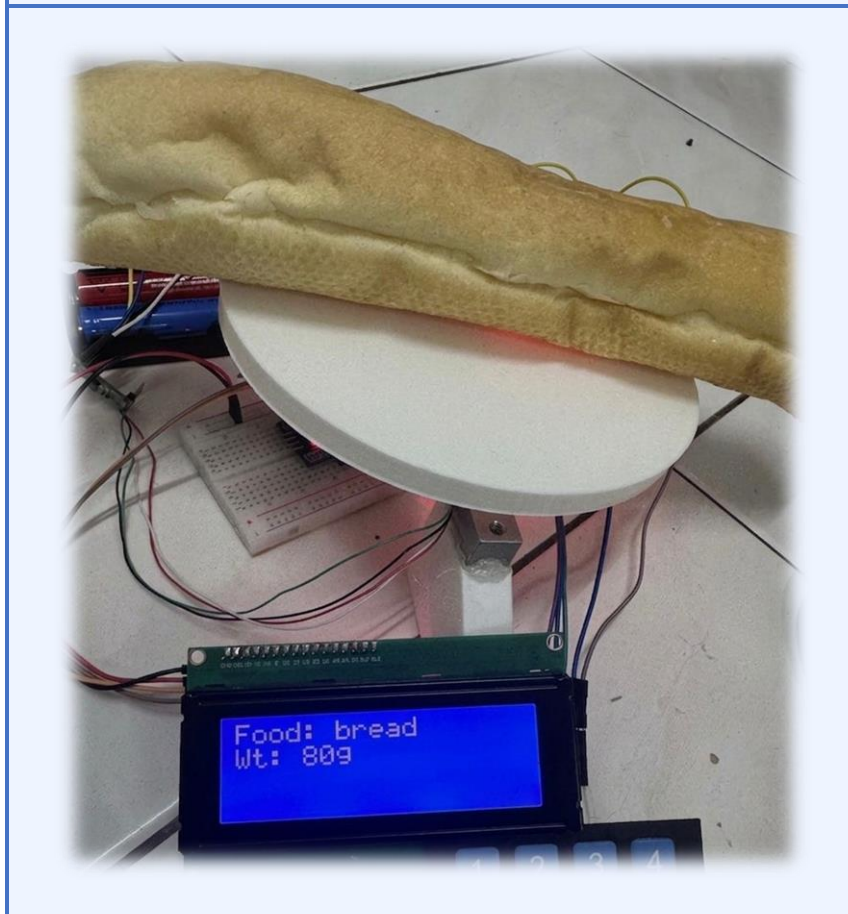
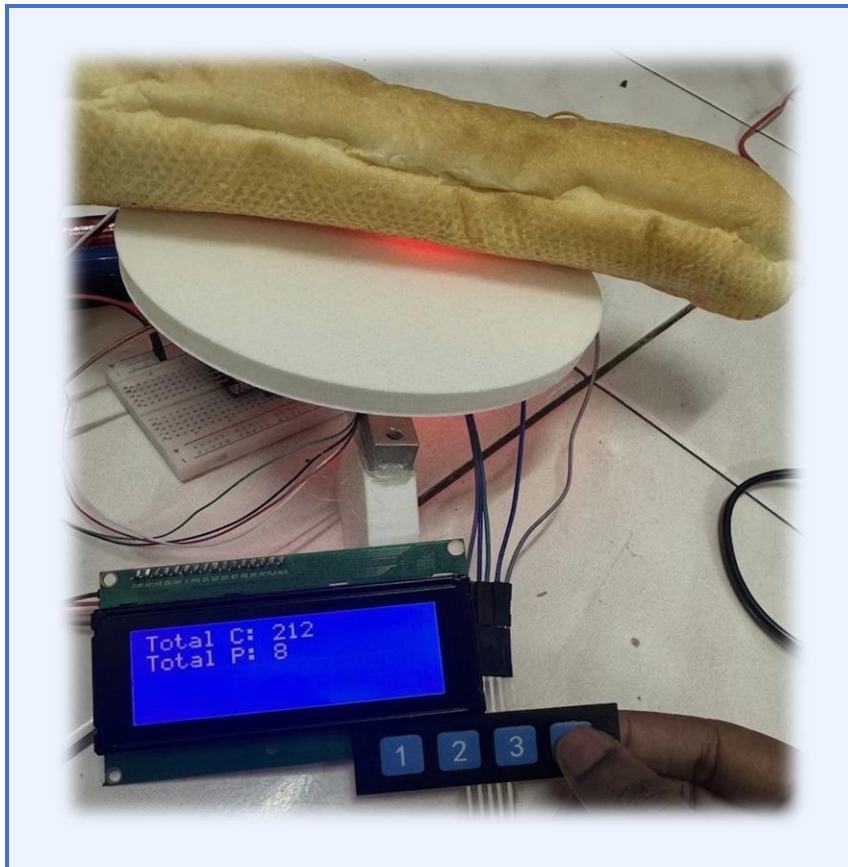
Four tactile push buttons are connected between their respective GPIO pins (D4–D7) and GND. PORTD internal pull-up resistors hold the lines at logic HIGH when buttons are open. Pressing a button pulls the pin LOW, which the firmware detects as active-low logic using the `BTN_x_PRESSED()` macros. No external resistors are required.

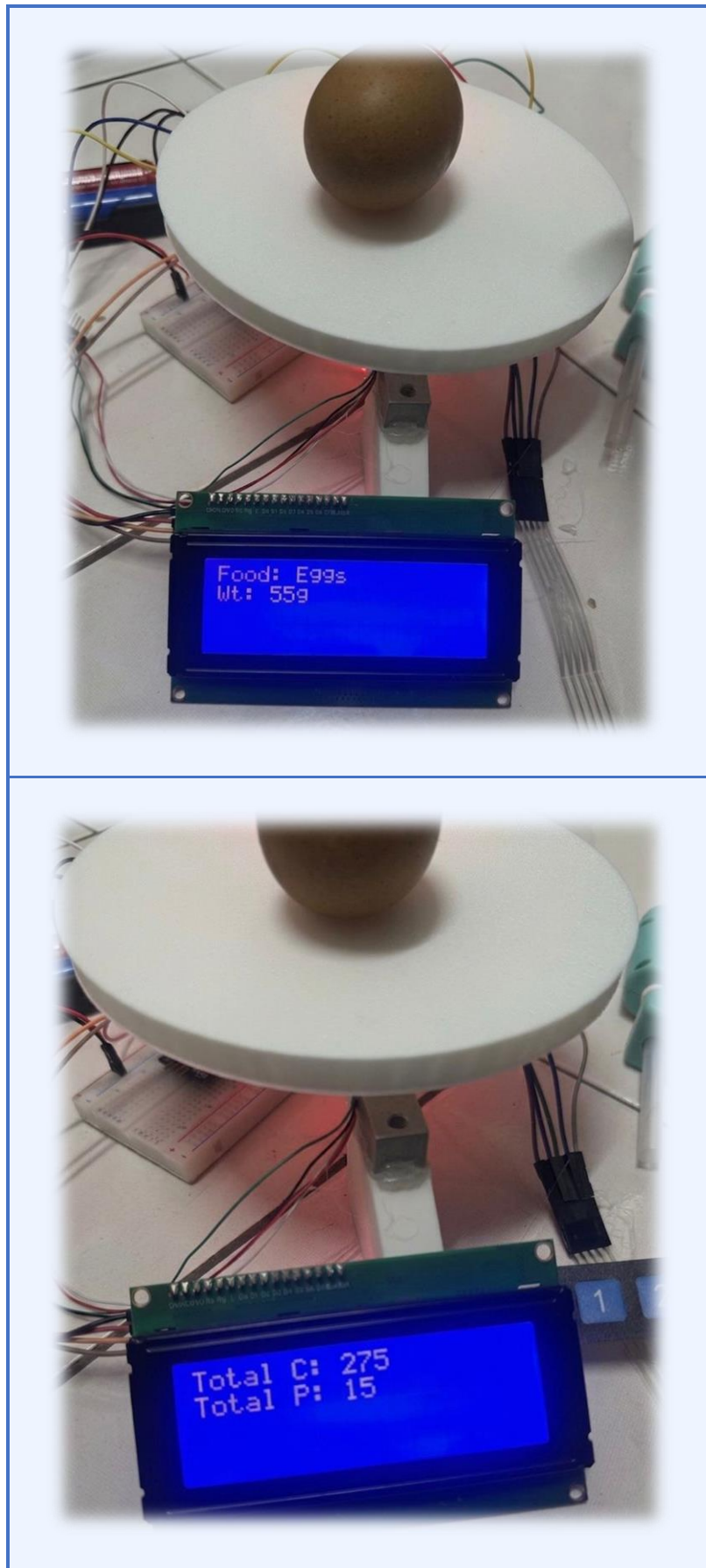
4.5 LED Indicator

The status LED (D1) is connected between PB0 and GND through a 330 Ω current-limiting resistor (R4). With a 5 V supply and a typical LED forward voltage of 2.0 V, the resistor limits current to approximately $(5 - 2) / 330 \approx 9$ mA — within the AVR pin sink/source limit of 40 mA.

4.6 Physical Prototype

After validating the circuit in simulation, a hardware prototype was assembled on a breadboard/PCB using the components listed above. Photographs of the assembled prototype are shown below.





5. Component Configuration

This section states the register-level and library-level configuration used for all active components in the design.

5.1 Timer 1 — CTC Mode (Millisecond Interrupt)

Timer 1 is configured in CTC (Clear Timer on Compare Match) mode to generate a precise 1 ms interrupt that drives the system time-base.

Pin	Direction	Function / Notes
D4 (PD4)	Input	BTN_CLEAR — Internal pull-up, active-low
D5 (PD5)	Input	BTN_VIEW — Internal pull-up, active-low
D6 (PD6)	Input	BTN_SCROLL — Internal pull-up, active-low
D7 (PD7)	Input	BTN_CONFIRM — Internal pull-up, active-low
B0 (PB0)	Output	Status LED — push-pull, initially LOW
D2	Input	HX711 DOUT (data)
D3	Output	HX711 SCK (clock)
A4/A5	I2C	SDA/SCL — PCF8574 I2C expander → LCD

Summary of Timer 1 register settings:

- TCCR1A = 0x00 — No waveform generation bits set (CTC uses WGM12 in TCCR1B only).
- TCCR1B = (WGM12 | CS11 | CS10) — CTC mode, prescaler = 64.
- OCR1A = 249 — Compare value for 1 ms period at 16 MHz / 64.
- TIMSK1 |= OCIE1A — Compare A match interrupt enabled.

5.2 GPIO Pin Configuration

All GPIO is configured through direct AVR register access. The table below summarises every pin used.

Pin	Dir.	Configuration & Purpose
D4–D7 (PD4–PD7)	Input	DDRD bits cleared; PORTD bits set — input with internal pull-up; active-low buttons.
B0 (PB0)	Output	DDRB bit 0 set; initialised LOW via PORTB — LED driver, push-pull output.
D2 (PD2)	Input	HX711 DOUT — digital serial data from ADC.
D3 (PD3)	Output	HX711 PD_SCK — clock signal to ADC.
A4 / A5	I2C	SDA / SCL for PCF8574 LCD expander — configured by Wire library.

5.3 HX711 Scale Calibration

- Calibration factor: 4.2 (set via `scale.set_scale(4.2)`).
- Reading mode: Average of 3 samples per reading to reduce noise.
- Tare: Performed at startup and via `BTN_CLEAR` to zero the output.
- RATE pin tied LOW: 10 SPS (samples per second) output data rate.

5.4 LCD Configuration

- Interface: I2C at 0x27 via PCF8574 expander.
- Dimensions: 16 columns × 2 rows.
- Backlight: Enabled at startup via `lcd.backlight()`.
- Character updates: Written using `lcd.setCursor()` and `lcd.print()` each main loop iteration.

6. Discussion — Subroutine Calls & Interrupt Execution

6.1 How the Interrupt Subroutine Works

The embedded system implements a hardware interrupt-driven time-base through Timer 1. When TIMSK1's OCIE1A flag is set, the AVR hardware automatically vectors to the ISR(TIMER1_COMPA_vect) routine every time Timer 1's counter matches OCR1A (every 1 ms). The interrupt context saves the current CPU registers on the stack, executes the ISR body (which simply increments `ms_ticks`), then restores registers and returns to wherever the main code was executing — completely transparently.

This approach decouples timekeeping from the main execution flow. The main loop can perform long operations (such as waiting for an HX711 reading or holding an LCD message) without missing timer counts, because the ISR runs preemptively regardless of what the main code is doing.

6.2 The `delay_ms()` Subroutine

The `delay_ms()` function is a user-defined subroutine that provides blocking delays by polling the volatile `ms_ticks` counter:

```
void delay_ms(uint32_t ms) {
    uint32_t start = ms_ticks;
    while ((ms_ticks - start) < ms);
}
```

When a button action calls `delay_ms(1000)`, execution stalls in the while loop for exactly 1000 ms. During this wait, the Timer 1 ISR continues to fire every 1 ms and increment `ms_ticks`, so the count remains accurate. The unsigned subtraction (`ms_ticks - start`) handles the 32-bit roll-over safely. This subroutine replaces the standard Arduino `delay()` with a register-level equivalent that the team fully controls.

6.3 Subroutine Call Sequence for Each Button

The table below traces which subroutines are called when each button is pressed:

Button	Subroutine / Action Sequence
BTN_CLEAR	LED_ON() → scale.tare() → lcd.clear() → lcd.print() → delay_ms(1000) → LED_OFF() → lcd.clear()
BTN_VIEW (short)	LED_ON() → lcd.clear() → lcd.print() [totals] → inner poll loop → delay_ms(4000) → LED_OFF() → lcd.clear()
BTN_VIEW (hold 2 s)	LED_ON() → lcd.clear() → lcd.print() [totals] → inner poll loop (≥2000 ms) → daily_cal=0, daily_pro=0 → delay_ms(1500) → LED_OFF() → lcd.clear()
BTN_SCROLL	LED_ON() → menu_idx++ (wrap) → delay_ms(300) → LED_OFF()

BTN_CONFIRM	LED_ON() → compute added_cal/added_pro → accumulate daily_cal/daily_pro → lcd.clear() → lcd.print() [saved] → delay_ms(4000) → LED_OFF() → lcd.clear()
-------------	--

In every action, LED_ON() and LED_OFF() serve as visual indicators that the MCU has entered and exited a subroutine block. The delay_ms() calls at the end of each action give the user time to read the LCD feedback message before the display reverts to the default weight/food view.

7. GitHub Repository

The full source code is attached to this github link

Repository URL: <https://github.com/mutazmudathir/MICRO-project.git>

8. Conclusion

The Smart Nutrition Scale project successfully demonstrates the integration of multiple embedded peripherals — a load-cell weight sensor, I2C LCD, four push buttons, and an LED — into a cohesive, functional embedded system programmed in C. The project met all requirements set out in the SEEE3223 group assignment specification.

The use of direct register-level GPIO configuration (DDRD, PORTD, PIND, DDRB, PORTB) rather than high-level library abstractions provided hands-on exposure to the AVR hardware layer. The Timer 1 CTC interrupt demonstrated how a hardware interrupt can drive a precise software time-base that operates independently of the main program flow — a fundamental pattern in real-time embedded systems.

The `delay_ms()` subroutine illustrated how a user-defined function can replace a library call while remaining compatible with the interrupt-driven time-base. The modular structure of the code — with separate `gpio_init()`, `timer1_init()`, and individual button-handler blocks — makes the firmware readable and extensible.

Through this project, the team gained practical experience in: AVR register manipulation, interrupt-driven programming, I2C peripheral interfacing, ADC-based sensor integration, and collaborative software development using GitHub. These skills are directly applicable to professional embedded systems engineering.

Future improvements could include persistent storage (EEPROM) for daily totals across power cycles, a larger food database, UART logging for meal history, and a more refined HX711 calibration procedure for higher accuracy.

Appendix A — Full Source Code

The complete firmware source code is listed below for reference.

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include "HX711.h"
#include <avr/io.h>
#include <avr/interrupt.h>

LiquidCrystal_I2C lcd(0x27, 16, 2);
HX711 scale;

const int DT_PIN = 2;
const int SCK_PIN = 3;

#define BTN_CLEAR_BIT    (1 << 4)
#define BTN_VIEW_BIT    (1 << 5)
#define BTN_SCROLL_BIT  (1 << 6)
#define BTN_CONFIRM_BIT (1 << 7)
#define LED_BIT          (1 << 0)

#define LED_ON()          (PORTB |=  LED_BIT)
#define LED_OFF()         (PORTB &= ~LED_BIT)
#define BTN_CLEAR_PRESSED() (! (PIND & BTN_CLEAR_BIT))
#define BTN_VIEW_PRESSED()  (! (PIND & BTN_VIEW_BIT))
#define BTN_SCROLL_PRESSED() (! (PIND & BTN_SCROLL_BIT))
#define BTN_CONFIRM_PRESSED() (! (PIND & BTN_CONFIRM_BIT))

volatile uint32_t ms_ticks = 0;

ISR(TIMER1_COMPA_vect) {
    ms_ticks++;
}

void timer1_init() {
    TCCR1A = 0;
    TCCR1B = (1 << WGM12) | (1 << CS11) | (1 << CS10);
    OCR1A = 249;
    TIMSK1 |= (1 << OCIE1A);
    sei();
}

void delay_ms(uint32_t ms) {
    uint32_t start = ms_ticks;
    while ((ms_ticks - start) < ms);
}

void gpio_init() {
    DDRD  &= ~(BTN_CLEAR_BIT|BTN_VIEW_BIT|BTN_SCROLL_BIT|BTN_CONFIRM_BIT);
    PORTD |= (BTN_CLEAR_BIT|BTN_VIEW_BIT|BTN_SCROLL_BIT|BTN_CONFIRM_BIT);
    DDRB |=  LED_BIT;
    PORTB &= ~LED_BIT;
}

typedef struct {
    char  name[10];
```

```
    float cal_per_g;
    float pro_per_g;
} FoodItem;

const int MENU_SIZE = 8;

FoodItem menu[MENU_SIZE] = {
    {"Chicken", 1.65, 0.31},
    {"Rice", 1.30, 0.03},
    {"Eggs", 1.55, 0.13},
    {"Beef", 2.50, 0.26},
    {"Broccoli", 0.34, 0.03},
    {"Oats", 3.89, 0.17},
    {"Whey Pro", 3.70, 0.78},
    {"Salmon", 2.08, 0.20}
};

int menu_idx = 0;
float daily_cal = 0;
float daily_pro = 0;
float current_weight = 0;

void setup() {
    gpio_init();
    timer1_init();
    lcd.init();
    lcd.backlight();
    lcd.print("System Booting..");
    scale.begin(DT_PIN, SCK_PIN);
    scale.set_scale(4.2);
    scale.tare();
    delay_ms(1000);
    lcd.clear();
}

void loop() {
    current_weight = scale.get_units(3);
    if (current_weight < 0) current_weight = 0;

    lcd.setCursor(0, 0);
    lcd.print("Food: ");
    lcd.print(menu[menu_idx].name);
    lcd.print(" ");
    lcd.setCursor(0, 1);
    lcd.print("Wt: ");
    lcd.print(current_weight, 0);
    lcd.print("g ");

    if (BTN_CLEAR_PRESSED()) {
        LED_ON();
        scale.tare();
        lcd.clear();
        lcd.print("Scale Cleared!");
        delay_ms(1000);
        LED_OFF();
        lcd.clear();
    }

    if (BTN_VIEW_PRESSED()) {
```

```
    LED_ON();
    lcd.clear();
    lcd.setCursor(0,0); lcd.print("Total C: "); lcd.print(daily_cal,0);
    lcd.setCursor(0,1); lcd.print("Total P: "); lcd.print(daily_pro,0);
    uint32_t hold_timer = 0; bool did_reset = false;
    while (BTN_VIEW_PRESSED()) {
        delay_ms(100); hold_timer += 100;
        if (hold_timer >= 2000) {
            lcd.clear(); lcd.setCursor(0,0);
            lcd.print("Totals Reset!");
            daily_cal = 0; daily_pro = 0;
            did_reset = true; delay_ms(1500); break;
        }
    }
    if (!did_reset) { delay_ms(4000); }
    LED_OFF(); lcd.clear();
}

if (BTN_SCROLL_PRESSED()) {
    LED_ON();
    menu_idx++;
    if (menu_idx >= MENU_SIZE) menu_idx = 0;
    delay_ms(300);
    LED_OFF();
}

if (BTN_CONFIRM_PRESSED()) {
    LED_ON();
    float added_cal = current_weight * menu[menu_idx].cal_per_g;
    float added_pro = current_weight * menu[menu_idx].pro_per_g;
    daily_cal += added_cal; daily_pro += added_pro;
    lcd.clear(); lcd.setCursor(0,0); lcd.print("Saved!");
    lcd.setCursor(0,1); lcd.print("C:"); lcd.print(daily_cal,0);
    lcd.print(" P:"); lcd.print(daily_pro,0);
    delay_ms(4000); LED_OFF(); lcd.clear();
}
}
```